

```
# Cell 1: Import thư viện và load model
```

```
import spacy
from spacy import displacy

# Load model tiếng Anh có parsing + vectors
nlp = spacy.load("en_core_web_md")

print("Loaded model:", nlp)
```

```
Loaded model: <spacy.lang.en.English object at 0x7a0f9d85a3f0>
```

```
# Cell 2: Phân tích câu ví dụ và in thông tin từng token
```

```
text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)

print("Câu:", text)
print(f"{'TEXT':<12} | {'POS':<8} | {'DEP':<10} | {'HEAD':<12}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<12} | {token.pos_:<8} | {token.dep_:<10} | {token.head.text:<12}")
```

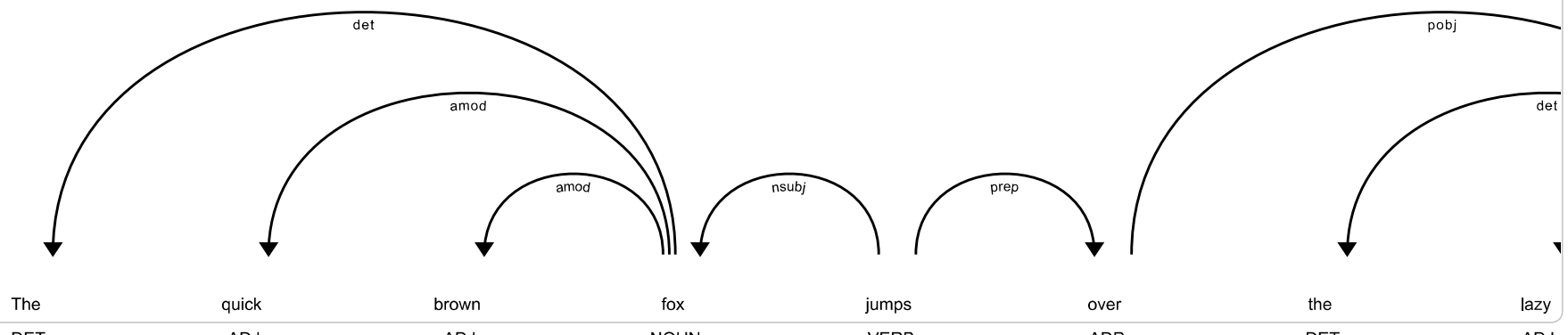
```
Câu: The quick brown fox jumps over the lazy dog.
```

TEXT	POS	DEP	HEAD
The	DET	det	fox
quick	ADJ	amod	fox
brown	ADJ	amod	fox
fox	NOUN	nsubj	jumps
jumps	VERB	ROOT	jumps
over	ADP	prep	jumps
the	DET	det	dog
lazy	ADJ	amod	dog
dog	NOUN	pobj	over
.	PUNCT	punct	jumps

```
# Cell 3: Trực quan hóa cây phụ thuộc
```

```
use_server = False # Đặt True nếu chạy local, False nếu chạy trong notebook/Jupyter

if use_server:
    displacy.serve(doc, style="dep") # Mở server (local)
else:
    displacy.render(doc, style="dep", jupyter=True) # Hiển thị trực tiếp trong notebook
```



Cell 4: Trả lời câu hỏi cho câu "The quick brown fox jumps over the lazy dog."

Tìm ROOT

```
root_token = [token for token in doc if token.dep_ == "ROOT"]
print("ROOT của câu là:", root_token[0].text if root_token else "Không tìm thấy")
```

Dependent của 'jumps'

```
for token in doc:
    if token.text.lower() == "jumps":
        deps = [(child.text, child.dep_) for child in token.children]
        print("\nCác dependent của 'jumps':")
        for word, dep in deps:
            print(f" - {word} ({dep})")
```

'fox' là head của những từ nào?

```
for token in doc:
    if token.text.lower() == "fox":
        children = [(child.text, child.dep_) for child in token.children]
        print("\nCác dependent của 'fox':")
        if children:
            for word, dep in children:
                print(f" - {word} ({dep})")
        else:
            print(" (Không có dependent)")
```

ROOT của câu là: jumps

Các dependent của 'jumps':

- fox (nsubj)
- over (prep)
- . (punct)

Các dependent của 'fox':

- The (det)
- quick (amod)
- brown (amod)

Cell 5: In bảng thông tin token cho câu Apple...

```
text2 = "Apple is looking at buying U.K. startup for $1 billion"
doc2 = nlp(text2)
```

```
print("Câu:", text2)
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | CHILDREN")
print("-" * 80)
```

```
for token in doc2:
    children = [child.text for child in token.children]
    print(f"{'token.text':<12} | {'token.dep_':<10} | {'token.head.text':<12} | {'token.head.pos_':<8} | {children}")
```

Câu: Apple is looking at buying U.K. startup for \$1 billion

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	[]
is	aux	looking	VERB	[]
looking	ROOT	looking	VERB	['Apple', 'is', 'at']
at	prep	looking	VERB	['buying']
buying	pcomp	at	ADP	['startup']
U.K.	compound	startup	NOUN	[]
startup	dobj	buying	VERB	['U.K.', 'for']
for	prep	startup	NOUN	['billion']
\$	quantmod	billion	NUM	[]
1	compound	billion	NUM	[]
billion	pobj	for	ADP	['\$', '1']

Cell 6: Tìm các bộ 3 (subject, verb, object)

```
text3 = "The cat chased the mouse and the dog watched them."
doc3 = nlp(text3)
```

```
print("Câu:", text3)
triplets = []
```

```
for token in doc3:
    # Chỉ xét các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""
        obj = ""
        # Tìm nsubj và dobj trong các con
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text
        if subject and obj:
            triplets.append((subject, verb, obj))
```

```
if triplets:
    print("\nCác bộ 3 (subject, verb, object) tìm được:")
    for s, v, o in triplets:
        print(f"  - ({s}, {v}, {o})")
else:
```

```
print("Không tìm thấy bộ 3 nào.")
```

Câu: The cat chased the mouse and the dog watched them.

Các bộ 3 (subject, verb, object) tìm được:

- (cat, chased, mouse)
- (dog, watched, them)

Cell 7: Tìm các tính từ bổ nghĩa cho danh từ

```
text4 = "The big, fluffy white cat is sleeping on the warm mat."
doc4 = nlp(text4)
```

```
print("Câu:", text4)
print()
```

```
for token in doc4:
    if token.pos_ == "NOUN":
        adjectives = []
        for child in token.children:
            if child.dep_ == "amod":
                adjectives.append(child.text)
        if adjectives:
            print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: {adjectives}")
```

Câu: The big, fluffy white cat is sleeping on the warm mat.

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']
 Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

Cell 8: Bài 1 – Tìm động từ chính của câu (ROOT)

```
def find_main_verb(doc):
    """
    Trả về token là động từ chính (ROOT) của câu.
    Nếu không thấy, trả về None.
    """
    for token in doc:
        if token.dep_ == "ROOT" and token.pos_ == "VERB":
            return token
    # Nếu ROOT không phải VERB (ví dụ: 'is', 'be', 'seem' hoặc câu danh từ)
    for token in doc:
        if token.dep_ == "ROOT":
            return token
    return None
```

Demo

```
example_sentences = [
    "The cat chased the mouse.",
    "Apple is looking at buying U.K. startup for $1 billion.",
    "Reading books helps you learn new things."
]
```

```
for s in example_sentences:
```

```
d = nlp(s)
main_verb = find_main_verb(d)
print(f"Câu: {s}")
print(" → Main verb (ROOT):", main_verb.text if main_verb else "Không tìm thấy")
```

Câu: The cat chased the mouse.
 → Main verb (ROOT): chased
 Câu: Apple is looking at buying U.K. startup for \$1 billion.
 → Main verb (ROOT): looking
 Câu: Reading books helps you learn new things.
 → Main verb (ROOT): helps

Cell 9: Bài 2 – Tự trích xuất Noun Chunks đơn giản

```
def extract_simple_noun_chunks(doc):
    """
    Trích xuất noun chunk đơn giản:
    - mỗi danh từ (NOUN)
    - cùng các từ bổ nghĩa: det, amod, compound (ở dạng children)
    Trả về list các chuỗi.
    """
    noun_chunks = []

    for token in doc:
        if token.pos_ == "NOUN":
            modifiers = []
            # Lấy các từ bổ nghĩa ở children
            for child in token.children:
                if child.dep_ in ("det", "amod", "compound"):
                    modifiers.append(child)
            # Sắp xếp theo vị trí trong câu
            all_tokens = modifiers + [token]
            all_tokens_sorted = sorted(all_tokens, key=lambda t: t.i)
            chunk_text = " ".join(t.text for t in all_tokens_sorted)
            noun_chunks.append(chunk_text)

    return noun_chunks

# Demo
text5 = "The big, fluffy white cat is sleeping on the warm mat near a small window."
doc5 = nlp(text5)

print("Câu:", text5)
print("Noun chunks (tự trích xuất):")
for chunk in extract_simple_noun_chunks(doc5):
    print("  -", chunk)

print("\nNoun chunks dùng spaCy built-in (để so sánh):")
for chunk in doc5.noun_chunks:
    print("  -", chunk.text)
```

Câu: The big, fluffy white cat is sleeping on the warm mat near a small window.
 Noun chunks (tự trích xuất):
 - The big fluffy white cat

- the warm mat
- a small window

Noun chunks dùng spaCy built-in (để so sánh):

- The big, fluffy white cat
- the warm mat
- a small window

Cell 10: Bài 3 - Tìm đường đi từ token bất kỳ đến ROOT

```
def get_path_to_root(token):
    """
    Trả về danh sách các token trên đường đi từ 'token' đến ROOT.
    Bao gồm chính token và token ROOT cuối cùng.
    """
    path = []
    current = token
    while True:
        path.append(current)
        if current.dep_ == "ROOT":
            break
        current = current.head
    return path

# Demo
text6 = "The big dog in the garden is barking loudly at the stranger."
doc6 = nlp(text6)

# Chọn một token, ví dụ 'stranger'
target = None
for t in doc6:
    if t.text.lower() == "stranger":
        target = t
        break

if target is not None:
    print("Câu:", text6)
    print("Token chọn:", target.text)
    path_tokens = get_path_to_root(target)
    print("Đường đi từ token đến ROOT:")
    print(" → " + " → " --> ".join(f'{t.text}({t.dep_})' for t in path_tokens))
else:
    print("Không tìm thấy token 'stranger' trong câu.")
```

```
Câu: The big dog in the garden is barking loudly at the stranger.
Token chọn: stranger
Đường đi từ token đến ROOT:
 → stranger(pobj) --> at(prej) --> barking(ROOT)
```

